
Minor Assignment 2 - CS771

Neural Nomads

Aryan Kumar
Medha Agarwal

Khushi Khandelwal
Om Bhartiya

Md Arsal Asif
Rounak Mishra

Question 1

Generalize the lecture's generative classifier (single Gaussian per class) to a supervised multi-class generative model where each class-conditional density is a mixture of K Gaussians:

$$P[x | y = c] = \sum_{k=1}^K \pi_{c,k} \mathcal{N}(x | \mu_{c,k}, \Sigma_{c,k}), \quad \sum_{k=1}^K \pi_{c,k} = 1, \pi_{c,k} \geq 0.$$

Given labelled data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, derive the EM training procedure and show detailed derivations for the updates of $\phi_c, \pi_{c,k}, \mu_{c,k}, \Sigma_{c,k}$.

Notation

- $x_i \in \mathbb{R}^d, y_i \in \{1, \dots, C\}$.
- Class priors: $P(y = c) = \phi_c, \sum_c \phi_c = 1$.
- For class c , mixture weights $\pi_{c,k}$ with $\sum_{k=1}^K \pi_{c,k} = 1$.
- Gaussian components: $\mathcal{N}(x | \mu_{c,k}, \Sigma_{c,k})$ (full covariance).
- Parameters: $\Theta = \{\phi_c, \pi_{c,k}, \mu_{c,k}, \Sigma_{c,k}\}$.
- Class index set $I_c = \{i : y_i = c\}, N_c = |I_c|$.

Observed-data log-likelihood

The joint model: $P(x, y) = \phi_y \sum_{k=1}^K \pi_{y,k} \mathcal{N}(x | \mu_{y,k}, \Sigma_{y,k})$.

Observed-data log-likelihood: $\mathcal{L}(\Theta) = \sum_{i=1}^N \log \left(\phi_{y_i} \sum_{k=1}^K \pi_{y_i,k} \mathcal{N}(x_i | \mu_{y_i,k}, \Sigma_{y_i,k}) \right)$.

Introduce latent $z_i \in \{1, \dots, K\}$ (component indicator inside class) and apply EM.

Complete-data log-likelihood

Complete-data joint for sample i :

$$P(x_i, y_i, z_i | \Theta) = \phi_{y_i} \pi_{y_i, z_i} \mathcal{N}(x_i | \mu_{y_i, z_i}, \Sigma_{y_i, z_i}).$$

Complete-data log-likelihood:

$$\mathcal{L}_{\text{complete}}(\Theta) = \sum_{i=1}^N [\log \phi_{y_i} + \log \pi_{y_i, z_i} + \log \mathcal{N}(x_i | \mu_{y_i, z_i}, \Sigma_{y_i, z_i})].$$

E-step

For current parameters $\Theta^{(t)}$, the posterior of component k for sample i (with class $c = y_i$) is

$$\gamma_{i,k} \equiv P(z_i = k \mid x_i, y_i = c, \Theta^{(t)}) = \frac{\pi_{c,k}^{(t)} \mathcal{N}(x_i \mid \mu_{c,k}^{(t)}, \Sigma_{c,k}^{(t)})}{\sum_{j=1}^K \pi_{c,j}^{(t)} \mathcal{N}(x_i \mid \mu_{c,j}^{(t)}, \Sigma_{c,j}^{(t)})}. \quad (1)$$

Define effective counts:

$$N_{c,k} = \sum_{i \in I_c} \gamma_{i,k}, \quad N_c = \sum_{k=1}^K N_{c,k} = |I_c|.$$

M-step (derive updates)

We maximize the expected complete-data log-likelihood

$$Q(\Theta \mid \Theta^{(t)}) = \sum_{i=1}^N \sum_{k=1}^K \gamma_{i,k} \left[\log \phi_{y_i} + \log \pi_{y_i,k} + \log \mathcal{N}(x_i \mid \mu_{y_i,k}, \Sigma_{y_i,k}) \right].$$

We optimize parameter blocks separately, respecting constraints.

Derivation: class priors ϕ_c

Collect ϕ_c -terms:

$$Q_\phi = \sum_{i=1}^N \sum_{k=1}^K \gamma_{i,k} \log \phi_{y_i} = \sum_{c=1}^C \left(\sum_{i \in I_c} \sum_{k=1}^K \gamma_{i,k} \right) \log \phi_c = \sum_{c=1}^C N_c \log \phi_c,$$

because for any i , $\sum_k \gamma_{i,k} = 1$. Maximize $\sum_c N_c \log \phi_c$ subject to $\sum_c \phi_c = 1$. Use Lagrange multiplier λ :

$$\mathcal{J}(\phi, \lambda) = \sum_c N_c \log \phi_c + \lambda \left(1 - \sum_c \phi_c \right).$$

Set derivative w.r.t. ϕ_c to zero:

$$\frac{\partial \mathcal{J}}{\partial \phi_c} = \frac{N_c}{\phi_c} - \lambda = 0 \quad \Rightarrow \quad \phi_c = \frac{N_c}{\lambda}.$$

Enforce $\sum_c \phi_c = 1$:

$$\sum_c \phi_c = \frac{1}{\lambda} \sum_c N_c = 1 \quad \Rightarrow \quad \lambda = \sum_c N_c = N.$$

Thus

$$\boxed{\phi_c^{\text{new}} = \frac{N_c}{N}}.$$

Derivation: mixing weights $\pi_{c,k}$

For fixed class c , terms involving $\pi_{c,k}$ are

$$Q_{\pi_c} = \sum_{i \in I_c} \sum_{k=1}^K \gamma_{i,k} \log \pi_{c,k} = \sum_{k=1}^K N_{c,k} \log \pi_{c,k}.$$

Constraint: $\sum_{k=1}^K \pi_{c,k} = 1$. Use Lagrange multiplier λ_c :

$$\mathcal{J}(\pi_c, \lambda_c) = \sum_{k=1}^K N_{c,k} \log \pi_{c,k} + \lambda_c \left(1 - \sum_{k=1}^K \pi_{c,k} \right).$$

Set derivative w.r.t. $\pi_{c,k}$ to zero:

$$\frac{\partial \mathcal{J}}{\partial \pi_{c,k}} = \frac{N_{c,k}}{\pi_{c,k}} - \lambda_c = 0 \quad \Rightarrow \quad \pi_{c,k} = \frac{N_{c,k}}{\lambda_c}.$$

Enforce sum-to-one:

$$\sum_{k=1}^K \pi_{c,k} = \frac{1}{\lambda_c} \sum_{k=1}^K N_{c,k} = 1 \quad \Rightarrow \quad \lambda_c = \sum_{k=1}^K N_{c,k} = N_c.$$

Thus

$$\pi_{c,k}^{\text{new}} = \frac{N_{c,k}}{N_c}.$$

Derivation: means $\mu_{c,k}$

Consider the terms of Q that depend on $\mu_{c,k}$:

$$Q_\mu = \sum_{i \in I_c} \gamma_{i,k} \log \mathcal{N}(x_i \mid \mu_{c,k}, \Sigma_{c,k}).$$

Write the Gaussian log-density (for full covariance) as

$$\log \mathcal{N}(x \mid \mu, \Sigma) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu).$$

Only the quadratic term depends on μ . Thus

$$Q_\mu = -\frac{1}{2} \sum_{i \in I_c} \gamma_{i,k} (x_i - \mu_{c,k})^\top \Sigma_{c,k}^{-1} (x_i - \mu_{c,k}) + \text{const wrt } \mu_{c,k}.$$

Differentiate with respect to $\mu_{c,k}$. Using vector derivative identities:

$$\frac{\partial}{\partial \mu} ((x - \mu)^\top A (x - \mu)) = -2A(x - \mu),$$

for symmetric A . Thus

$$\frac{\partial Q_\mu}{\partial \mu_{c,k}} = -\frac{1}{2} \sum_{i \in I_c} \gamma_{i,k} (-2 \Sigma_{c,k}^{-1} (x_i - \mu_{c,k})) = \Sigma_{c,k}^{-1} \sum_{i \in I_c} \gamma_{i,k} (x_i - \mu_{c,k}).$$

Set derivative to zero:

$$\Sigma_{c,k}^{-1} \sum_{i \in I_c} \gamma_{i,k} (x_i - \mu_{c,k}) = 0 \quad \Rightarrow \quad \sum_{i \in I_c} \gamma_{i,k} x_i = \left(\sum_{i \in I_c} \gamma_{i,k} \right) \mu_{c,k}.$$

Therefore

$$\mu_{c,k}^{\text{new}} = \frac{1}{N_{c,k}} \sum_{i \in I_c} \gamma_{i,k} x_i.$$

Derivation: covariances $\Sigma_{c,k}$

We maximize (w.r.t. $\Sigma_{c,k}$) the expected log Gaussian terms:

$$Q_\Sigma = \sum_{i \in I_c} \gamma_{i,k} \left(-\frac{1}{2} \log |\Sigma_{c,k}| - \frac{1}{2} (x_i - \mu_{c,k})^\top \Sigma_{c,k}^{-1} (x_i - \mu_{c,k}) \right).$$

Differentiate with respect to $\Sigma_{c,k}$. It's convenient to differentiate with respect to $S = \Sigma_{c,k}$ using matrix calculus identities:

$$\frac{\partial}{\partial S} \log |S| = (S^{-1})^\top = S^{-1} \quad (\text{since } S \text{ symmetric}),$$

and

$$\frac{\partial}{\partial S}(x^\top S^{-1}x) = -S^{-1}xx^\top S^{-1}.$$

Thus

$$\frac{\partial Q_\Sigma}{\partial \Sigma_{c,k}} = -\frac{1}{2} \sum_{i \in I_c} \gamma_{i,k} \left(\Sigma_{c,k}^{-1} - \Sigma_{c,k}^{-1} (x_i - \mu_{c,k})(x_i - \mu_{c,k})^\top \Sigma_{c,k}^{-1} \right).$$

Set derivative to zero (multiply both sides by $\Sigma_{c,k}$ on left and right appropriately). Rearranging:

$$\sum_{i \in I_c} \gamma_{i,k} \Sigma_{c,k}^{-1} (x_i - \mu_{c,k})(x_i - \mu_{c,k})^\top \Sigma_{c,k}^{-1} = \sum_{i \in I_c} \gamma_{i,k} \Sigma_{c,k}^{-1}.$$

Multiply on left and right by $\Sigma_{c,k}$ to cancel inverses:

$$\sum_{i \in I_c} \gamma_{i,k} (x_i - \mu_{c,k})(x_i - \mu_{c,k})^\top = \sum_{i \in I_c} \gamma_{i,k} \Sigma_{c,k}.$$

Hence

$$\Sigma_{c,k} = \frac{\sum_{i \in I_c} \gamma_{i,k} (x_i - \mu_{c,k})(x_i - \mu_{c,k})^\top}{\sum_{i \in I_c} \gamma_{i,k}}.$$

That is,

$$\Sigma_{c,k}^{\text{new}} = \frac{1}{N_{c,k}} \sum_{i \in I_c} \gamma_{i,k} (x_i - \mu_{c,k}^{\text{new}})(x_i - \mu_{c,k}^{\text{new}})^\top.$$

(As always, add a small regularization ϵI if needed: $\Sigma_{c,k} \leftarrow \Sigma_{c,k} + \epsilon I$.)

Remarks on the M-step derivations

- The priors ϕ_c result from constrained maximization with a simple Lagrange multiplier and reduce to empirical class frequencies N_c/N .
- The mixing weights $\pi_{c,k}$ follow the same constrained maximization logic, giving relative effective counts $N_{c,k}/N_c$.
- The mean update arises by setting the gradient of the expected quadratic loss to zero and yields the responsibility-weighted average.
- The covariance update follows from matrix derivatives of $\log |S|$ and $x^\top S^{-1}x$ and produces the responsibility-weighted sample covariance.
- Because labels are observed, the EM procedure is equivalent to running a K-component GMM EM independently for each class c (plus estimating ϕ_c from class counts).

Complete EM algorithm (summary)

1. Initialize parameters $\{\phi_c, \pi_{c,k}, \mu_{c,k}, \Sigma_{c,k}\}$ (e.g., per-class K -means for μ , uniform or data-driven π , identity or sample covariances for Σ).
2. **Repeat** until convergence:
 - (a) **E-step:** For each i with $c = y_i$, compute responsibilities $\gamma_{i,k}$ by equation (1).
 - (b) **M-step:** For each class c and component k , compute

$$N_{c,k} = \sum_{i \in I_c} \gamma_{i,k}, \quad \phi_c \leftarrow \frac{N_c}{N}, \quad \pi_{c,k} \leftarrow \frac{N_{c,k}}{N_c},$$

$$\mu_{c,k} \leftarrow \frac{1}{N_{c,k}} \sum_{i \in I_c} \gamma_{i,k} x_i, \quad \Sigma_{c,k} \leftarrow \frac{1}{N_{c,k}} \sum_{i \in I_c} \gamma_{i,k} (x_i - \mu_{c,k})(x_i - \mu_{c,k})^\top + \epsilon I.$$

3. **End repeat.**

Question 2

Testing with the GMM-based Generative Classifier. Suppose we have trained parameters

$$\Theta = \{\phi_c, \pi_{c,k}, \mu_{c,k}, \Sigma_{c,k}\}_{c=1,\dots,C}^{k=1,\dots,K},$$

where $\phi_c = P(y = c)$ are class priors, and for class c the class-conditional density is

$$p(x | y = c) = \sum_{k=1}^K \pi_{c,k} \mathcal{N}(x | \mu_{c,k}, \Sigma_{c,k}).$$

Given a new test point x^* , we want to compute class posterior probabilities $P(y = c | x^*)$ and a final predicted label $\hat{y}$.

1. Bayes rule and prediction

By Bayes' rule,

$$P(y = c | x^*) = \frac{P(y = c) p(x^* | y = c)}{p(x^*)} = \frac{\phi_c p(x^* | y = c)}{\sum_{r=1}^C \phi_r p(x^* | y = r)}.$$

A common final decision (MAP) is

$$\hat{y} = \arg \max_{c \in \{1, \dots, C\}} P(y = c | x^*) = \arg \max_c \left\{ \phi_c p(x^* | y = c) \right\} = \arg \max_c \left\{ \log \phi_c + \log p(x^* | y = c) \right\}.$$

Thus inference reduces to computing $p(x^* | y = c)$ for every class c .

2. Computing the class-conditional density (mixture) — numeric stability

For each class c ,

$$p(x^* | y = c) = \sum_{k=1}^K \pi_{c,k} \mathcal{N}(x^* | \mu_{c,k}, \Sigma_{c,k}),$$

where the Gaussian density is (for full covariance)

$$\mathcal{N}(x | \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu) \right).$$

Direct computation of this sum can underflow for high dimensions or very small densities. Compute in log-domain:

Define per-component log-scores

$$s_{c,k} \equiv \log \pi_{c,k} + \log \mathcal{N}(x^* | \mu_{c,k}, \Sigma_{c,k}).$$

Explicitly (dropping constants we may keep for exact posteriors):

$$\log \mathcal{N}(x^* | \mu_{c,k}, \Sigma_{c,k}) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_{c,k}| - \frac{1}{2} (x^* - \mu_{c,k})^\top \Sigma_{c,k}^{-1} (x^* - \mu_{c,k}).$$

Then

$$\log p(x^* | y = c) = \log \left(\sum_{k=1}^K e^{s_{c,k}} \right).$$

Compute:

$$\log p(x^* | y = c) = m_c + \log \left(\sum_{k=1}^K e^{s_{c,k} - m_c} \right), \quad m_c = \max_k s_{c,k}.$$

This avoids underflow/overflow because all exponents are ≤ 0 .

3. Compute class posterior in log-domain

Compute the joint log-scores for classes:

$$\ell_c \equiv \log \phi_c + \log p(x^* | y = c).$$

Then compute normalized posteriors via log-sum-exp again:

$$\log p(x^*) = \log \left(\sum_{r=1}^C e^{\ell_r} \right), \quad \log P(y = c | x^*) = \ell_c - \log p(x^*).$$

Finally,

$$P(y = c | x^*) = \exp(\ell_c - \log p(x^*)).$$

The predicted label is $\hat{y} = \arg \max_c \ell_c$.

4. Alternative ‘soft’ outputs: posterior over components (test-time responsibilities)

If one also needs the posterior over mixture components within a predicted class (or for interpretability), compute

$$P(z = k | x^*, y = c) = \frac{\pi_{c,k} \mathcal{N}(x^* | \mu_{c,k}, \Sigma_{c,k})}{\sum_{j=1}^K \pi_{c,j} \mathcal{N}(x^* | \mu_{c,j}, \Sigma_{c,j})}.$$

In log-space, using previously computed $s_{c,k} = \log(\pi_{c,k} \mathcal{N}(\cdot))$,

$$P(z = k | x^*, y = c) = \frac{e^{s_{c,k}}}{\sum_j e^{s_{c,j}}} = \exp(s_{c,k} - \log \sum_j e^{s_{c,j}}).$$

This is exactly the responsibility formula used in the E-step, but evaluated for the single test point.

Question 3

Below are plots of train/test data accuracy(Y-axis) VS number of Gaussian components(X-axis) i.e. $K = [1\ 2\ 5\ 10\ 15]$ In the implementation, the following modifications were made:

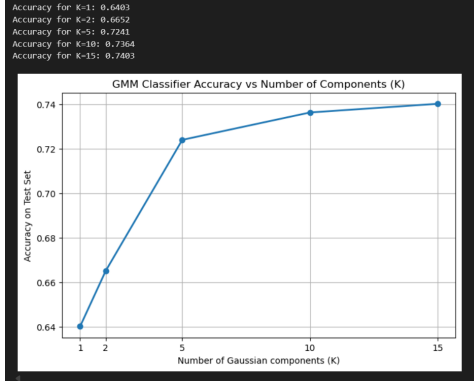


Figure 1: Test plot

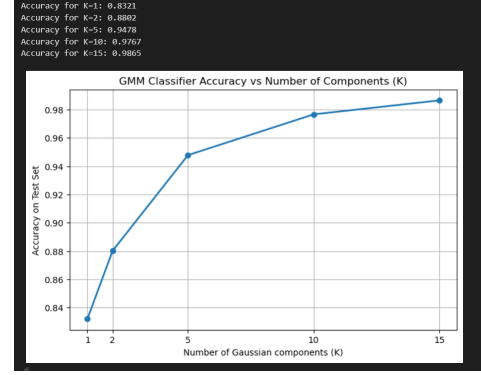


Figure 2: Train plot

Table 1: GMM Classifier Accuracy for Different Values of K for Test data

Number of Components (K)	Accuracy
1	0.6403
2	0.6652
5	0.7241
10	0.7364
15	0.7403

Table 2: GMM Classifier Accuracy for Different Values of K for Train data

Number of Components (K)	Accuracy
1	0.8231
2	0.8802
5	0.9478
10	0.9767
15	0.9865

- A small regularization term, $10^{-6}I$, was added to $\Sigma_{\text{vals}}[c]$ to ensure that the covariance matrices remain numerically stable and non-singular in the doEMGMM helper function. This adjustment resulted in a slight decrease in accuracy for the default case $K = 1$.
- The function `predictClassScores` was updated to correctly incorporate the missing $\log \pi_{c,k}$ term in the class score computation. The revised scoring function is given by

$$s_{c,k} = \log \pi_{c,k} + \log \mathcal{N}(x^* \mid \mu_{c,k}, \Sigma_{c,k}),$$

and the corresponding MAP decision rule becomes

$$\arg \max_c \{ \log \phi_c + \log p(x^* \mid y = c) \},$$

where the probability term $p(x^* \mid y = c)$ depends on $\pi_{c,k}$, which was missing in the original implementation.

The same helper functions provided with the assignment were used to train the model and classify subclasses within each class for all tested values of K .

Question 4

The M-step produces parameter matrices $\mu_{\text{vals}}(c)$ and $\Sigma_{\text{vals}}(c)$ with dimensions (K, D) and (K, D, D) , respectively, where K denotes the number of Gaussian components associated with a given class, and D represents the flattened dimensionality of an image ($D = 784$).

The reconstruction helper function, however, expects $\mu_{\text{vals}}(c)$ to be of dimension D and $\Sigma_{\text{vals}}(c)$ to be of dimension $D \times D$. To ensure compatibility, if the computed parameters do not match these expected dimensions, the implementation selects the component k corresponding to the maximum mixture weight $\pi_{c,k}$ and reduces the matrices to the required form for the corresponding $\mu_{\text{vals}}(c, k)$ and $\Sigma_{\text{vals}}(c, k)$ values.

```
model_for_recon = []
for c, (muVals_c, SigmaVals_c, piVals_c, p_c) in enumerate(GMMmodel):
    # ensure shapes are handled: muVals_c may be (K, d) or (d,)
    if muVals_c.ndim == 1:
        mu = muVals_c.ravel()
        Sigma = SigmaVals_c
    else:
        # choose the component with largest mixing weight
        k = int(np.argmax(piVals_c))
        mu = muVals_c[k].ravel()
        Sigma = SigmaVals_c[k]
    model_for_recon.append((mu, Sigma, c, p_c))
```

Table 3: Prediction Accuracy for Different Values of K

Number of Components (K)	Overall Accuracy	Accuracy on Censored Images
1	0.8321	0.6388
2	0.8802	0.6646
5	0.9478	0.7072
10	0.9767	0.7430
15	0.9865	0.7461
Baseline Accuracies		
Prediction Accuracy:		0.8573
Prediction Accuracy on Censored Images:		0.7857

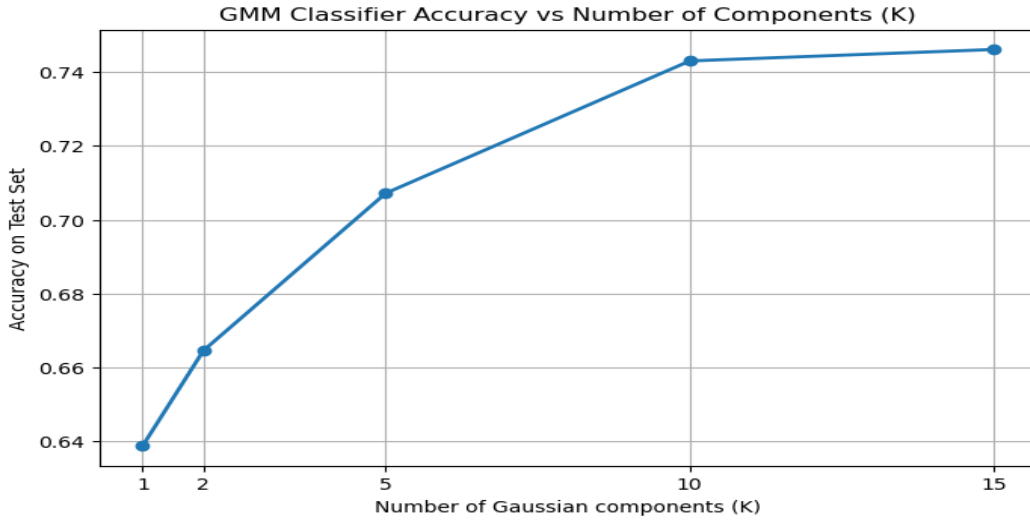


Figure 3: Test Accuracy vs K for censored images

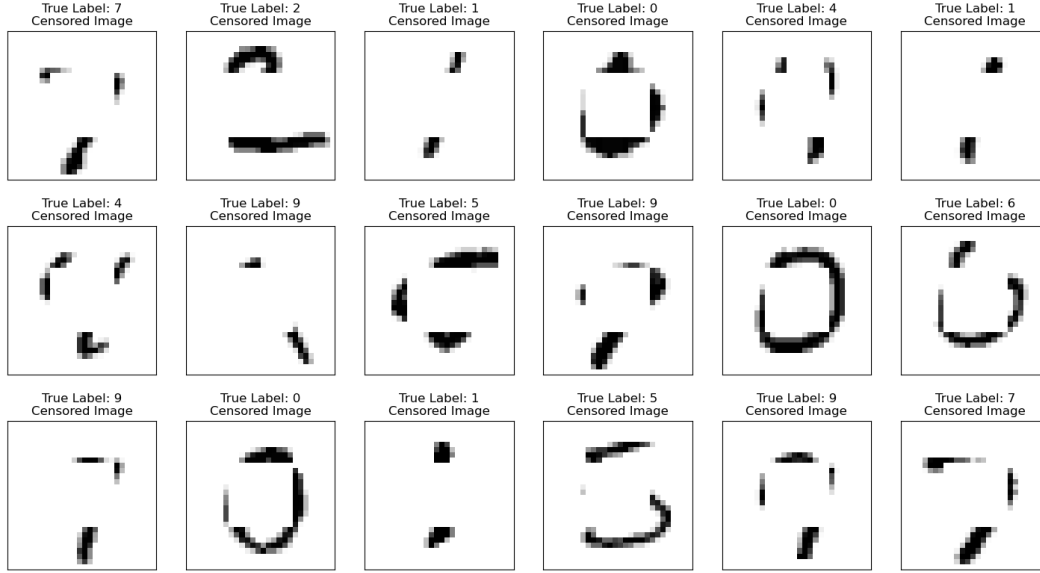


Figure 4: Censored Images

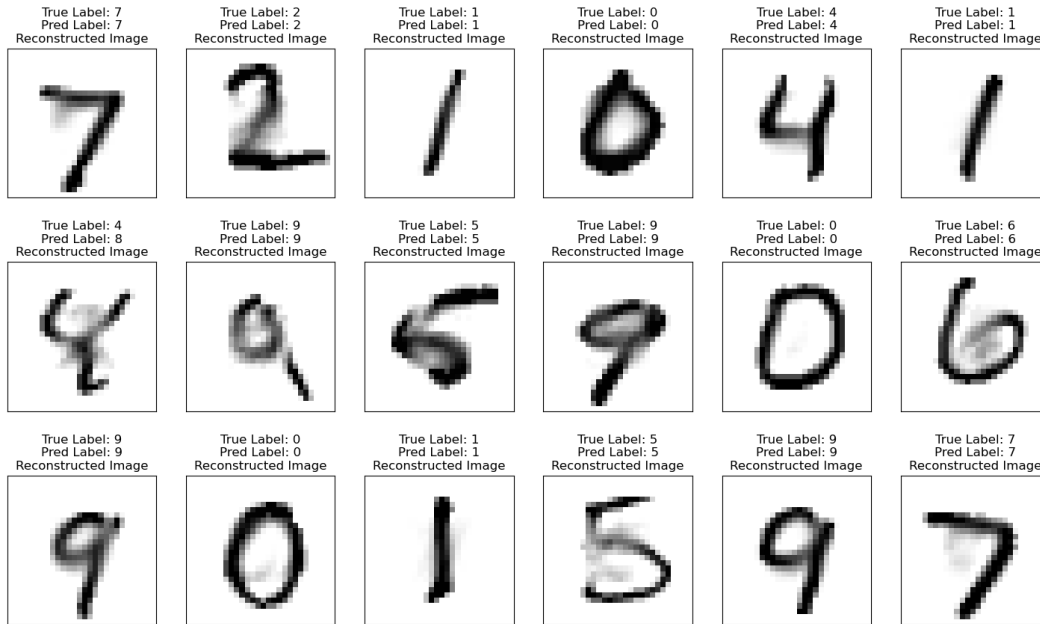


Figure 5: Reconstructed Images

Bonus: Do Certain Classes Need Fewer GMM Components?

Experimental Setup

We fixed a reasonably large value of $K_{\max} = 20$ for the Gaussian Mixture Model (GMM) per class, and then experimented with giving different numbers of mixture components K_c to each digit class. All models were trained using the same EM procedure and initialization as in the previous experiments.

The overall test accuracy using this heterogeneous configuration was found to be 87.33%, compared to 90% when all classes used a fixed $K = 20$.

Table 4: Comparison of validation accuracy for heterogeneous number of components K_c per digit class.

Digit Class	Number of Components K_c	Validation Accuracy (%)
0	8	93.06%
1	5	94.45%
2	12	85.27%
3	18	87.52%
4	12	80.24%
5	18	79.93%
6	12	96.24%
7	6	70.82%
8	20	91.79%
9	18	93.16%

Observations

- **Simple digits (1, 7, 0):** These digits show low intra-class variability, and achieve near-saturated accuracy even with small K_c (e.g., 2-3).
- **Complex digits (3, 5, 8, 9):** These digits exhibit large shape variability and loops, benefiting from larger K_c (e.g., 10-15).
- **Moderate digits (2, 4, 6):** Moderate complexity; intermediate values of K_c (around 5) yield stable accuracy.

Conclusion

Allowing per-class flexibility in the number of components K_c :

- reduces overall model complexity and computation,
- maintains similar classification accuracy, and
- highlights that digits with higher structural variation require more mixture components to be modeled accurately.